

React JWT Authentication Best Practices Guide

The **best and most secure way to handle JWT token authentication in React** is to store your short-lived `accessToken` **in-memory (React State)** and store your `refreshToken` in a secure, **HttpOnly cookie**.

Storing tokens in `localStorage` or `sessionStorage` makes your application vulnerable to **Cross-Site Scripting (XSS)** attacks, where malicious scripts can steal your user's tokens.

Standard React Hooks for JWT Setup

When setting up JWT authentication and calling APIs, a combination of three standard hooks is used:

Hook	Purpose in JWT Authentication
<code>useContext</code>	Used to create a global custom hook (e.g., <code>useAuth</code>) that shares the login state and the in-memory <code>accessToken</code> across all application components.
<code>useEffect</code>	Used to trigger a silent token refresh (via the HttpOnly cookie) when the app first loads or when a user refreshes their browser tab.
<code>useMemo</code> / Custom Hook	Usually wrapped into a custom hook like <code>useAxiosPrivate</code> . It sets up an API client (like <code>Axios</code>) with interceptors that automatically attach the JWT to outgoing requests and handle 401 expiration retries.

Step-by-Step Architecture Blueprint

1. The Global Auth Context (`useAuth`)

```
import { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [accessToken, setAccessToken] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const verifyUserSession = async () => {
      try {
        const response = await myPublicApi.post('/refresh');
        setAccessToken(response.data.accessToken);
      } catch (err) {
        console.error("Session expired / No valid refresh token");
      } finally {
        setLoading(false);
      }
    }
  });
};
```

```

    verifyUserSession();
  }, []));

  return (
    <AuthContext.Provider value={{ accessToken, setAccessToken, loading }}>
      {!loading && children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);

```

2. The Private API Client Hook (useAxiosPrivate)

```

import { useEffect } from 'react';
import { privateAxiosInstance } from './apiClient';
import { useAuth } from './useAuth';
import { useRefreshToken } from './useRefreshToken';

export const useAxiosPrivate = () => {
  const { accessToken } = useAuth();
  const refresh = useRefreshToken();

  useEffect(() => {
    const requestIntercept = privateAxiosInstance.interceptors.request.use(
      config => {
        if (!config.headers['Authorization']) {
          config.headers['Authorization'] = `Bearer ${accessToken}`;
        }
        return config;
      },
      error => Promise.reject(error)
    );

    const responseIntercept = privateAxiosInstance.interceptors.response.use(
      response => response,
      async (error) => {
        const prevRequest = error?.config;
        if (error?.response?.status === 401 && !prevRequest?.sent) {
          prevRequest.sent = true;
          const newAccessToken = await refresh();
          prevRequest.headers['Authorization'] = `Bearer ${newAccessToken}`;
          return privateAxiosInstance(prevRequest);
        }
        return Promise.reject(error);
      }
    );

    return () => {
      privateAxiosInstance.interceptors.request.eject(requestIntercept);
      privateAxiosInstance.interceptors.response.eject(responseIntercept);
    };
  }, [accessToken, refresh]);

  return privateAxiosInstance;
};

```

3. Consuming Inside a Component

```

import { useEffect, useState } from 'react';
import { useAxiosPrivate } from '../hooks/useAxiosPrivate';

```

```
export const Dashboard = () => {
  const [data, setData] = useState(null);
  const axiosPrivate = useAxiosPrivate();

  useEffect(() => {
    axiosPrivate.get('/protected-dashboard-data')
      .then(res => setData(res.data))
      .catch(err => console.error(err));
  }, [axiosPrivate]);

  return <div>{data ? JSON.stringify(data) : "Loading..."}</div>;
};
```